

FLUTTER / DART / PROJE MİMARİSİ VE AI DESTEKLİ KİŞİSEL ÇALIŞMA PLANI UYGULAMASI

Bu doküman, Flutter, Dart, Backend, API ve Machine Learning taraflarını sınav öncesi hızlı tekrar yapabilecek şekilde hazırlanmıştır.

BÖLÜM 1 - DART VE FLUTTER TEMELLERİ

1.1 Dart Nedir?

- Dart, Google tarafından geliştirilmiş nesne yönelimli (OOP) bir programlama dilidir.
- Flutter uygulamalarının tamamı Dart ile yazılır.

Temel özellikleri:

- Object Oriented Programming
- Null Safety
- Async/Await desteği
- Mobil, Web ve Desktop geliştirme

Sınav Sorusu:

Dart nedir?

Cevap:

Dart, Flutter uygulamaları geliştirmek için kullanılan Google tarafından geliştirilmiş nesne yönelimli bir programlama dilidir.

1.2 Flutter Nedir?

- Flutter bir programlama dili değildir.
- Flutter bir UI Framework'tür.
- Dart dili ile çalışır.

Amaç:

Tek kod tabanı ile;

- Android
- IOS
- Windows

- Web

uygulamaları geliştirebilmektir.

Sınav Sorusu:

Flutter nedir?

Cevap:

Flutter, Dart dili kullanılarak mobil, web ve masaüstü uygulamaları geliştirmek için kullanılan açık kaynaklı bir UI framework'üdür.

1.3 Dart ve Flutter İlişkisi

Dart = Programlama Dili

Flutter = Framework

İlişki:

Dart

↓

Flutter

↓

Mobil Uygulama

BÖLÜM 2 - FLUTTER'IN ÇALIŞMA MANTIĞI

2.1 Widget Nedir?

Flutter'da her şey widget'tır.

Örnek:

- Text
- Button
- Icon
- Container
- Column
- Row
- Scaffold

Örnek:

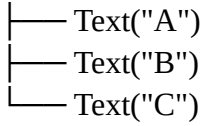
Text("Merhaba Dünya")

2.2 Parent - Child Mantığı

Flutter'ın en önemli mimari yapısıdır.

Örnek:

Column



Burada:

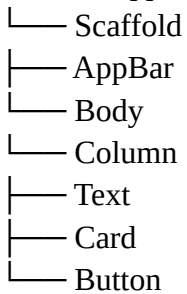
- Column = Parent
- Text = Child

2.3 Widget Ağacı

Flutter ekranları aslında ağaç yapısıdır.

Örnek:

MaterialApp



Sınavda bu yapıyı açıklayabilmek önemlidir.

2.4 Stateless ve Stateful Widget

StatelessWidget

- Durumu değişmeyen ekranlar
- Hakkımızda
- Yardım
- Bilgilendirme ekranları

StatefulWidget

- Durumu değişebilen ekranlar
- Form ekranları
- Sayaçlar
- Test giriş ekranları

BÖLÜM 3 - FLUTTER'DA KATMANLI MİMARİ

Tipik Flutter yapısı:

lib/

├── models/

├── providers/

├── services/

├── screens/

├── widgets/

└── main.dart

3.1 Models

Veri taşıyan sınıflardır.

Örnek:

- User
- TestResult
- StudyPlan
- SubjectGoal

Görev:

Veriyi taşımak.

3.2 Services

Dış sistemlerle iletişim kurar.

Örnek:

- ApiService
- LocalStorageService

Görevleri:

- API çağrıları
- Yerel veri işlemleri

3.3 Provider

State Management katmanıdır.

Görevi:

Uygulamanın ortak belleğini yönetmek.

Örnek:

- Dersler
- Testler
- Hedefler
- Tema bilgileri

3.4 Screens

Kullanıcının gördüğü ekranlardır.

Örnek:

- HomeScreen
- AnalysisScreen
- PlanScreen
- SettingsScreen

3.5 Widgets

Tekrar kullanılabilir UI bileşenleridir.

Örnek:

- CustomButton
- SubjectCard
- StatCard

BÖLÜM 4 - STATE MANAGEMENT

Provider Neden Kullanılır?

Küçük projelerde:

setState()

kullanılabilir.

Büyük projelerde:

- Provider
- Riverpod
- Bloc

tercih edilir.

Bu projede Provider kullanılmıştır.

Çalışma Mantığı

Veri değiştiğinde:

notifyListeners();

çalışır.

Ardından ilgili ekranlar yeniden çizilir.

Sınav Sorusu:

Provider neden kullanılır?

Cevap:

Provider, uygulamanın ortak durumunu merkezi şekilde yönetmek ve veri değişikliklerinde ekranların otomatik güncellenmesini sağlamak için kullanılır.

BÖLÜM 5 - YEREL VERİ SAKLAMA

Hive Nedir?

Hive, Flutter tarafında kullanılan NoSQL veri tabanıdır.

Görevi:

Verileri cihazda saklamak.

Saklanan veriler:

- Dersler
- Test sonuçları
- Çalışma planları
- Kullanıcı bilgileri

Avantajı:

Uygulama kapansa bile veriler silinmez.

Sınav Sorusu:

Firestore yerine neden Hive kullanıldı?

Cevap:

İnternet bağlantısı olmadan çalışabilmek ve verileri cihaz üzerinde saklamak amacıyla Hive tercih edilmiştir.

BÖLÜM 6 - PROJENİN GENEL MİMARİSİ

Bu proje üç temel parçadan oluşmaktadır.

Flutter

↓

FastAPI

↓

Machine Learning

Detaylı akış:

Kullanıcı

↓

Flutter

↓

HTTP Request

↓

FastAPI

↓

ML Modeli

↓

Tahmin

↓

Flutter

BÖLÜM 7 - PROJENİN AMACI

Amaç:

Öğrencinin akademik performansını analiz ederek kişiselleştirilmiş çalışma planı oluşturmak.

Sistem:

- Test sonuçlarını toplar
- Mevcut seviyeyi belirler
- Hedefe uzaklığı hesaplar
- Çalışma önerileri üretir

BÖLÜM 8 - MODELİN KULLANDIĞI VERİLER

Inputlar:

subject

total_questions

correct

wrong
study_time
difficulty
current_net
target_net

Açıklamalar:

subject → Ders adı
total_questions → Toplam soru
correct → Doğru sayısı
wrong → Yanlış sayısı
study_time → Çalışma süresi
difficulty → Zorluk seviyesi
current_net → Mevcut net
target_net → Hedef net

Output:

predicted_net

BÖLÜM 9 - BACKEND KISMI

FastAPI Nedir?

Python ile API geliştirmek için kullanılan framework'tür.

Bu projede Flutter ile ML modeli arasında köprü görevi görmektedir.

Endpointler

/predict

Amaç:

Tahmini net hesaplamak.

Input:

```
{  
  "subject": "math",  
  "current_net": 20  
}
```

Output:

```
{  
  "predicted_net": 31  
}
```

/plan

Amaç:

En uygun çalışma planını oluşturmak.

Çalışma Mantığı:

100 farklı plan üret

↓

Modele gönder

↓

Sonuçları karşılaştır

↓

En iyi planı seç

BÖLÜM 10 - MACHINE LEARNING KISMI

Kullanılan Kütüphaneler

- pandas
- scikit-learn
- joblib

Dataset Üretimi

Gerçek öğrenci verisi olmadığı için:

produce.py

ile sentetik veri üretilmiştir.

Toplam:

100.000 kayıt

Train.py

Görevleri:

- Dataseti okur
- Eğitim verisini hazırlar
- Modeli eğitir
- Modeli kaydeder

Random Forest Regressor

Kullanılan algoritma:

RandomForestRegressor

Neden Random Forest?

- Regresyon problemlerinde başarılıdır
- Overfitting riski düşüktür
- Karmaşık ilişkileri öğrenebilir
- Kullanımı kolaydır

Sınav Sorusu:

Neden Random Forest kullandınız?

Cevap:

Regresyon problemlerinde yüksek doğruluk sağlaması, karmaşık ilişkileri öğrenebilmesi ve overfitting riskinin düşük olması nedeniyle tercih edilmiştir.

BÖLÜM 11 - MODEL.PKL DOSYASI

model.pkl Nedir?

Eğitilmiş modelin kaydedilmiş halidir.

Kaydetme:

```
joblib.dump(model)
```

Yükleme:

```
joblib.load("model.pkl")
```

Avantajı:

Model her açılışta yeniden eğitilmez.

BÖLÜM 12 - MAE (MEAN ABSOLUTE ERROR)

MAE:

Modelin ortalama hata miktarını ölçer.

Örnek:

$MAE = 0.95$

Anlamı:

Model ortalama olarak yaklaşık 0.95 net hata yapmaktadır.

Sınav Sorusu:

MAE neyi ifade eder?

Cevap:

Model tahminlerinin gerçek deęerlerden ortalama ne kadar saptıęını gsteren hata metrięidir.

BÖLÜM 13 - AI KISMININ ÇALIŞMA MANTIĞI

Bu projede ChatGPT benzeri üretken yapay zeka kullanılmamıştır.

Kullanılan yaklaşım:

Makine Öğrenmesi (Regression)

+

Optimizasyon

+

Kural Tabanlı Destek

Tahmin Mekanizması

Veri Al

↓

DataFrame Oluştur

↓

One Hot Encoding

↓

Model Predict

↓

Tahmini Net

Plan Üretimi

100 farklı çalışma süresi üret

↓

Her birini modele sor

↓

En yüksek neti veren planı seç

↓

Kullanıcıya göster

Bu nedenle sistem yalnızca tahmin yapan deęil, karar desteęi saęlayan bir AI sistemi olarak deęerlendirilebilir.

BÖLÜM 14 - UYGULAMA EKRANLARI

Dashboard

- Mevcut net

- Hedef net
- Ortalama net
- Test sayısı
- Önerilen süre

Test Giriş Ekranı

- Ders
- Toplam soru
- Doğru
- Yanlış
- Süre
- Zorluk

Backend'e gönderilir.

Analiz Ekranı

- Ortalama net
- Hedef açığı
- Toplam test
- Zayıf dersler

Plan Ekranı

- Önerilen çalışma süresi
- Tahmini net
- Net açığı

Rapor Ekranı

- Ders bazlı performans
- Çalışma süresi grafiği
- Net gelişimi

Ayarlar

- Tema değişikliği
- Hedef güncelleme
- Veri sıfırlama

BÖLÜM 15 - KULLANILAN FLUTTER PAKETLERİ

provider → State Management

http → API iletişimi

hive → Local Database

hive_flutter → Flutter entegrasyonu

path_provider → Hive dosya yolu

fl_chart → Grafik çizimi

intl → Tarih işlemleri

cupertino_icons → Icon desteği

BÖLÜM 16 - SINAV İÇİN TEK CÜMLELİK ÖZET

Bu projede Flutter kullanılarak mobil arayüz geliştirildi, Provider ile uygulama durumu yönetildi, Hive ile veriler cihazda saklandı, FastAPI üzerinden Python tabanlı backend ile iletişim kuruldu ve Random Forest regresyon modeli kullanılarak öğrencilerin performans tahmini ve çalışma planı önerileri üretildi.